

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C# programlama dili, aynı zamanda modüler bir programlama dilidir. Oluşturulan ve önceden test edilip doğrulanmış modüller olan **sınıf kütüphaneleri** (**class library**) yazacağımız kodlara **using** yönergesi ile dahil edilir.

using Kütüphane-Adı; //Bu yönergeler de talimattır ve noktalı virgül ile biter.

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağrılması, işlemciyi yorar. Çünkü her fonksiyon çağrısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak modern derleyiciler **satır içi genişletme** (**JIT Inlining**) yöntemiyle **küçük fonksiyonları doğrudan ana koda gömerler ve böylece bu yükü sıfıra indirirler**.

Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C# programlama dilinde en çok kullanacağımız modüllerden biri de giriş çıkış IO (**input output**) işlemlerinin tanımlı olduğu **System**¹ ve **System.IO**² sınıf kütüphaneleridir. Bu kütüphaneler, yeni proje şablonunda **örtük olarak koda dahil edilmiştir**. Böylece kodumuza ek bir **using** yönergesi yazmadan konsoldan bir şey okumak ve konsola bir şey yazmak için **Console** sınıfını doğrudan kullanırız. Bunların yanında **Math**, **Random**, **string** gibi birçok sınıf kullanır hale geliriz.

```
/* Bu kütüphaneler örtük olarak tanımlıdır!
using System;
using System.IO;
*/
Console.WriteLine("Merhaba "); // Konsola "Merhaba" yazar ve imleci son yazılan karakter sonrasına
→ taşır.
```

5.2 Konsola Veri Yazma

Konsoldan veri okumak ve konsola veri yazmak için **Console** sınıfı kullanılır. Sınıfın **Write** yöntemi ile imlecin (**cursor**) bulunduğu yerden itibaren parametre olarak girilen veriler konsola yazılır ve imleç son yazılan karakterden sonrasına taşınır. Sınıfın **WriteLine** yöntemi kullanılarak veriler yazıldıktan sonra son yazılan karakterden sonra imleç bir alt satıra inerek satır başı yapar.

```
Console.WriteLine("Merhaba ");
Console.WriteLine("İlhan");
int yaş = 50;
Console.WriteLine(yaş);
float kilo = 99.5f;
Console.WriteLine(kilo);
decimal para = 545.5M;
Console.WriteLine(para);
/*Program Çıktısı:
Merhaba İlhan
50
99,5
545,5
*/
```

Write ve **WriteLine** yönteminin ilk argümanı metin olmak zorunda değildir. Ancak bu yöntemlerin ilk argümanı, **biçimlendirme metni** (**format string**) olarak kullanılabilir;

¹<https://learn.microsoft.com/en-us/dotnet/api/system>

²<https://learn.microsoft.com/en-us/dotnet/api/system.io>

- ◊ Biçimlendirme metninin içinde süslü parantez kullanılarak izleyen argümanların indisi verilir.
- ◊ İzleyen her bir argümanların indisi sıfırdan başlar ve sırasıyla devam eder.
- ◊ Metin içindeki süslü parantezle belirtilen yere ilgili argümanın değeri yazılır.
- ◊ Son olarak oluşturulan metin konsola yazılır.

```

1 // See https://aka.ms/new-console-template for more information
2 int sayi1 = 20, sayi2 = 30;
3 Console.WriteLine("1.Sayılar:{0}, {1}",sayi1,sayi2);
4 int toplam = sayi1 + sayi2;
5 Console.WriteLine("2.Sayıların Toplamı:{0}", toplam);
6 Console.WriteLine("3.Toplamı {0} olan sayılar: {1}, {2}", toplam, sayi1, sayi2);
7 Console.WriteLine("4.Toplam: {0}, toplam({0}) şeklinde de yazılabilir", toplam);
8 /*Program Çıktısı:
9 1.Sayılar:20, 30
10 2.Sayıların Toplamı:50
11 3.Toplamı 50 olan sayılar: 20, 30
12 4.Toplam: 50, toplam(50) şeklinde de yazılabilir
13 */

```

Bu kodu şu şekilde açıklayabiliriz;

- ◊ 3. Satırdaki `WriteLine("1.Sayılar:{0}, {1}",sayi1,sayi2);` talimatı ile `"1.Sayılar:{0}, {1}"` metnindeki `{0}` görülen yere izleyen **ilk argümanın değeri** konur, ardından `{1}` görülen yere izleyen **ikinci argümanın değeri** konur. Sonuç olarak oluşan metin `"1.Sayılar:20, 30"` olacaktır. Bu durumda sonuç metin konsola yazdırılır ve imleç bu metin yazdırıldıktan sonra satır başı yaptırılır.
- ◊ 5. Satırdaki `WriteLine("2.Sayıların Toplamı:{0}", toplam);` talimatı ile `"2.Sayıların Toplamı:{0}"` metnindeki `{0}` görülen yere izleyen **ilk argümanın değeri** konur. Sonuç olarak oluşan metin, konsola yazdırılacak metindir ve `"2.Sayıların Toplamı: 50"` şeklinde olacaktır. Bu durumda sonuç metin konsola yazdırılır ve imleç bu metin yazdırıldıktan sonra satır başı yaptırılır.
- ◊ 6. satırda da benzer şekilde bir konsola yazdırma yapılacaktır.
- ◊ 7. satırda da benzer şekilde bir konsola yazdırma yapılacaktır. Burada, aynı indisteki argümanın biçimlendirme metni içerisinde istenildiği yere, istenildiği kadar ve istenildiği sırada konulduğu gözden kaçırılmamalıdır.

5.3 Metinler

Metinleri (**string**) kod içerisinde işlemek için aslında karakter dizileri kullanılır. Ancak, daha yaygın uygulama olarak bir metin değişkeni bildirmek için **string** anahtar sözcüğünü kullanılır. Bu anahtar kelime, **System.String** sınıfı için bir takma addır.

§5.3.1 Metinleri Tanımlama

Metin değişmezleri (**string literal**) kod içerisinde **çift tırnak** (") karakterleri arasında yazılır. Aslında bu metinler UTF-16 karakterlerinden oluşur. C# dilinde karakterler için **char** veri tipi kullanılır. Karakter değişmezleri (**character literal**) ise **tek tırnak** (') arasında yazılır;

```

1 string adı, soyadı;
2 adı = "İlhan";
3 soyadı = "ÖZKAN";
4 Console.WriteLine("Adı ve Soyadı : {0} {1}", adı, soyadı);
5 char karakter1 = 'h';
6 char karakter2 = '\u006A';// Tek tırnak içinde UTF-16 kodlamasına göre
7 char karakter3 = '\x0013';
8 char karakter4 = 'Ş';
9 Console.WriteLine("{0} {1} {2} {3}", karakter1, karakter2, karakter3, karakter4);
10 char[] harfler = { 'A', 'B', 'C', 'Ç', 'D', 'E', 'F' }; //Karakter Dizisi
11 Console.WriteLine(harfler); //Her bir dizi elemanı ayrı ayrı yazılır!
12 string selamlama = new string(harfler); //Karakter disizi metne çevrildi (cast)
13 Console.WriteLine("Alfabe: {0}", selamlama);
14 /*Program Çıktısı:
15 Adı ve Soyadı : İlhan ÖZKAN
16 h j Ş
17 ABCÇDEF
18 Alfabe: ABCÇDEF
19 */

```

Metin değişmezleri içinde çift tırnak karakteri koymak için (\") yazılır. Benzer şekilde tek tırnak karakteri değişmezi için de tek tırnak arasında (') yazılır;

```
/*
Metin değişmezler çift tırnak arasında yazılır.
Metin değişmezi içinde çift tırnak kullanmak için \" olarak yazılır.
Bir sonraki satıra geçmek için \ karakteri kullanılır.
Metin değişmezi içinde \ karakteri kullanmak için \\ yazılır.
*/
string metin = "Merhaba \"İlhan\"";
Console.WriteLine(metin);
/*Program Çıktısı:
Merhaba "İlhan"
*/
```

§5.3.2 Tam Metin Değişmezleri

Bir önceki örnekte metin değişmezinde birçok kural vardır. Onları gözardı etmek için **tam metin işleci** (@) kullanılır. Metnin başına bu işleç konularak **tam metin** (**verbatim string**) değişmesi tanımlanabilir. Tam metnin içinde sadece çift tırnak karakteri için iki adet çift tırnak karakteri (""") kullanılır;

```
string filename1 = @"c:\documents\files\u0066.txt"; // Tam metin (verbatim string)
string filename2 = "c:\\documents\\files\\u0066.txt"; // Sıratan metin (string)
Console.WriteLine(filename1);
Console.WriteLine(filename2);
string tammetin= @"İlhan
Özkan \ """;
Console.WriteLine(tammetin);
/*Program Çıktısı:
c:\documents\files\u0066.txt
c:\documents\files\u0066.txt
İlhan
Özkan \ "
*/
```

§5.3.3 Temel Metin İşlemleri

Metinlerin karakter sayısı, aynı zamanda **uzunluğudur**. Metin uzunluğu için **Length** özelliğini kullanırız;

```
string str = "İlhan Özkan";
int length = str.Length;
Console.WriteLine(length); //11
```

Metinler, **String.Format()** yöntemini kullanarak biçimlendirme metni (**format string**) ile bir veya daha fazla öğeyi tek bir metinde birleştirebiliriz. Burada ilk argüman biçimlendirme metnidir. **Console.Write()** yöntemi gibi çalışır. Konsola yazdırılan metin gibi biçimlendirilmiş yeni bir metin oluşturur;

```
string yenimetin=string.Format("Yaşım: {0} ve Adım: {1}", 45, "İlhan");
Console.WriteLine(yenimetin); //Yaşım:45 ve Adım:İlhan
```

Bunların yanında metinler toplama işleci (+) ile birleştirilebilir. Bunun için **String.Concat()**, **String.Join()** ve **String.Format()** yöntemleri de kullanılabilir;

```
string s1 = "İlhan";
string s2 = "Özkan";
string s3 = s1 + " " + s2; //"İlhan Özkan"
string s4 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s5 = string.Join(" ", s1, s2); //"İlhan Özkan"
string s6 = string.Format("{0} {1}", s1, s2); //"İlhan Özkan"
string s7 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s8 = string.Join(" ", new string[] { s1, s2 }); //"İlhan Özkan"
string s9 = $"{s1} {s2}"; //"İlhan Özkan" //Enterpolasyon İleride anlatılacaktır!
```

Metinler içerisindeki UTF-8 karakterleri 4 karakterli onaltılık sayı olarak da verilebilir;

```
string s = "\u0130lhan"; //"İlhan"
```

Metinlere ilk değer çift tırnak karakterleri içerisinde verilir. Peki metin çift tırnak karakteri içeriyorsa ters bölü işareti ile birlikte çift tırnak karakteri (\") kullanılır. **Kaçış dizileri** (**escape sequences**), metinsel ifadeler (**string**) içinde **doğrudan yazılamayan** veya **özel bir anlam taşıyan karakterleri** temsil etmek için

kullanılır. Bunlar Tablo 5.1’da verilmiştir. C# dilinin Unicode desteği, çok dilli uygulamalar geliştirmedeki gücünü gösterir.

Dizi	Karakter Adı	Unicode	Açıklama / Örnek
'	Tek Tırnak	0x0027	Karakter sabitlerinde kullanılır.
"	Çift Tırnak	0x0022	Metin (string) içinde tırnak yazdırmak için.
\\	Ters Bölü	0x005C	Dosya yollarında yaygın kullanılır.
\0	Boş (null) karakter	0x0000	Bellek sonlandırıcı karakter.
\a	Uyarı (alert)	0x0007	Sistem bip sesini tetikler.
\b	Geri (backspace)	0x0008	İmleci bir karakter geri kaydırır.
\e	Kaçış (escape)	0x001B	ESC karakteri.
\f	Yeni Sayfa (form feed)	0x000C	Sayfa başı (yazıcılar için).
\n	Yeni Satır (new line)	0x000A	İmleci bir alt satırın başına indirir.
\r	Satır Başı (carriage return)	0x000D	İmleci mevcut satırın başına döndürür.
\t	Yatay sekme (horizontal tab)	0x0009	Yatayda bir sekme, 8 karakter ve katlarına kadar boşluk bırakır.
\v	Dikey Sekme (vertical tab)	0x000B	Dikey sekme, aynı sütunda alt satıra imleci taşır.
\u	Unicode (UTF-16)	\uHHHH	4 haneli onaltılık kod: \u00E7 (ç)
\U	Unicode (UTF-32)	\U00HHHHHH	8 haneli onaltılık kod: \U0000015E (Ş)
\x	Onaltılık Karakter	\xH[H][H][H]	Değişken uzunluklu onaltılık kod. \x015E = \x15E (Ş)

Tablo 5.1: Kaçış Dizileri ve Unicode Karşılıkları

Ayrıca metni belirlenen bir aralıkta sağa ya da sola yaslama (**padding**) da yapılabilir;

```
string s = "\u0130lhan '61'";
Console.WriteLine("\u0130lhan '61'"); //İlhan '61'
string paddedLeft = s.PadLeft(10, ' '); //Sola boşluk ekle
Console.WriteLine(paddedLeft); // İlhan
string paddedRight = s.PadRight(10, '+'); //Sağa + karakteri ekle
Console.WriteLine(paddedRight); //İlhan+++++
```

Bit tam sayıyı da metin olarak herhangi bir tabana **Convert()** yöntemi çevirebiliriz;

```
int32 Number = 252;
Console.WriteLine(Convert.ToString(Number, 2)); //11111000
Console.WriteLine(Convert.ToString(Number, 8)); //370
Console.WriteLine(Convert.ToString(Number, 10)); //252
Console.WriteLine(Convert.ToString(Number, 16)); //FC
```

Değer tipler de **ToString()** yöntemi ile metne çevrilebilir. Burada metne dönüştürülecek veri tipine özel biçimlendirme metni içinde çeşitli karakterler (0,d,M,y,H,m,s) olduğuna dikkat ediniz;

```
int intValue = 31;
Console.WriteLine(intValue.ToString("000")); //031
Console.WriteLine(intValue.ToString("Değer 0 dır.")). //Değer 31 dır.
double doubleValue = 10.456;
Console.WriteLine(doubleValue.ToString("0.00")); //10,46
Console.WriteLine(doubleValue.ToString("00")); //10
Console.WriteLine(doubleValue.ToString("Değer 0.0 dır.")). //Değer 10.5 dır.
DateTime currentDate = DateTime.Now; // 14/05/2025 10:38:03
Console.WriteLine(currentDate.ToString("dd-MM-yyyy HH:mm:ss")); // "14-05-2025 10:38:03"
Console.WriteLine(currentDate.ToString("dd-MM-yyyy")); // "14-05-2025"
Console.WriteLine(currentDate.ToString("dd-MMMM-yyyy HH:mm:ss")); // "14-Mayıs-2025 10:38:03"
```

§5.3.4 Metin Enterpolasyonu

Metin enterpolasyonu (**string interpolation**), bir metin ifadesinin başına dolar (\$) işleci konulduğunda, o metin artık **enterpolasyonlu metin** olur.

Enterpolasyon İfadeleri

Enterpolasyon işleci (\$) sayesinde, metin içinde süslü parantezler kullanılarak;

1. Matematik işlemleri veya mantıksal işlemler kodlanabilir.
2. Doğrudan kapsamdaki (scope) değişkenler, isimleri ile kodlanabilir.

```
//METİN BİRLEŞTİRME:
string ad = "Mustafa";
int yas = 25;
//1. Klasik Yöntem (+ operatörü ile)
string metin1 = "Merhaba " + ad + ", yaşıınız " + yas + ".";
//2. String.Format Yöntemi
string metin2 = string.Format("Merhaba {0}, yaşıınız {1}.", ad, yas);
//3. MODERN YÖNTEM (String Interpolation)
string metin3 = $"Merhaba {ad}, yaşıınız {yas}.";
Console.WriteLine(metin3); //Merhaba Mustafa, yaşıınız 25.

//MATEMATİKSEL İŞLEMLER:
int sayi1 = 10, sayi2 = 20;
Console.WriteLine($"Toplam: {sayi1 + sayi2}"); //Toplam: 30
Console.WriteLine($"Sonuç: {(sayi1 > sayi2 ? "Büyük" : "Küçük")}"); //Toplam: 30

double fiyat = 1250.50;
Console.WriteLine($"Fiyat: {fiyat:C2}"); // Para birimi (Currency) -> $1.250,50
DateTime bugun = DateTime.Now;
Console.WriteLine($"Tarih: {bugun:dd/MM/yyyy}"); // 11/04/2026

Eğer metniniz hem kaçış dizilerini (ters bölü gibi) içermesin hem de enterpolasyon özelliğini kullansın istiyorsanız, tam metin işleci ($) ile birlikte metin enterpolasyonu işlecini (@) birleştirerek ($@) kullanabilirsiniz;

// See https://aka.ms/new-console-template for more information
string klasor = "Kullanıcılar";
string yol = @$"C:\{klasor}\Belgelerim";
Console.WriteLine(yol); //C:\Kullanıcılar\Belgelerim
```

Metin Enterpolasyonu

C#6.0 ile gelen metin enterpolasyonu, arka planda **string.Format** fonksiyonuna dönüştürülür.

§5.3.5 String Sınıfı

Programlama dünyasının en temel taşlarından biri olan **String** sınıfı, metinsel veriler üzerinde işlem yapmamızı sağlayan devasa bir araç kutusu gibidir. İster Java, ister C# kullanıyor olun; bu yöntemler mantık olarak birbirine çok benzer.

Bilgi Alma Yöntemleri

Bu yöntemler, metnin içeriğini değiştirmeden size metin hakkında bilgi verir.

- ◊ **length()**: Metnin toplam karakter sayısını (uzunluğunu) döndürür.
- ◊ **charAt(index)**: Belirtilen indisteki karakteri verir. (Unutmayın, indisler her zaman 0'dan başlar!)
- ◊ **indexOf(keyword)**: Aranılan bir karakterin veya alt metnin başladığı ilk indisi verir. Bulamazsa -1 döner.

Metin Düzenleme ve Değiştirme

String'ler genellikle değiştirilemez (immutable) olduğu için, bu yöntemler mevcut metni değiştirmek yerine size yeni bir metin döndürür.

- ◊ **toLowerCase()** / **toUpperCase()**: Tüm harfleri küçük veya büyük harfe çevirir.
- ◊ **trim()**: Metnin başındaki ve sonundaki gereksiz boşlukları temizler.
- ◊ **replace(eski, yeni)**: Metin içindeki belirli bir parçayı başka bir parçayla değiştirir.
- ◊ **substring(başlangıç, bitiş)**: Metnin belirli bir aralığını kesip almanızı sağlar.

Kontrol ve Karşılaştırma

Metnin içeriğinde ne olduğunu sorgulamak için kullanılır.

- ◇ `contains(sequence)`: Metnin içinde belirli bir kelime veya harf dizisi geçip geçmediğini kontrol eder (true/false).
- ◇ `startsWith(prefix)` / `endsWith(suffix)`: Metnin belirli bir ifadeyle başlayıp başlamadığını kontrol eder.
- ◇ `equals(otherString)`: İki metnin birbirine tıpatıp aynı olup olmadığını kontrol eder. (Büyük/küçük harf duyarlıdır).

Metni Parçalara Ayırma

Özellikle veri işleme süreçlerinde hayat kurtarır. `split(delimiter)`: Metni belirli bir ayırıcıya (örneğin virgül veya boşluk) göre bölerek bir diziye (`array`) dönüştürür. Eğer performansın kritik olduğu ve sürekli metin birleştirdiğiniz bir döngüdeyseniz, standart `String` yerine `StringBuilder` sınıfını kullanmak uygulamınızı ciddi anlamda hızlandıracaktır.

5.4 Konsoldan Veri Okuma

Konsoldan veri okuma, kullanıcı ile program arasındaki etkileşimin ilk adımıdır. C# dilinde bu işlem için `System` isim uzayının (`namespace`) altındaki `Console` sınıfı kullanılır.

§5.4.1 Temel Okuma Yöntemler

Konsoldan veri okumak için üç ana yöntemimiz vardır;

1. `string? ReadLine()`; yöntemi, kullanıcı **tamam** (**enter**) tuşuna basana kadar yazdığı tüm satırı okur.
2. `int Read()`; yöntemi, kullanıcının girdiği ilk karakterin ASCII/Unicode sayısal değerini döndürür.
3. `ConsoleKeyInfo ReadKey()`; yöntemi, kullanıcının bastığı tek bir tuşu (Ctrl, Alt, Harf vb.) yakalar. Genellikle programı bekletmek için kullanılır.

§5.4.2 Veri Okuma ve Tip Dönüşümü

`Console.ReadLine()` her zaman bir `string` döndürür. Eğer kullanıcıdan sayısal bir veri alıp üzerinde matematiksel işlem yapacaksanız, bu veriyi uygun türe dönüştürmeniz (**casting**) şarttır. Bunun için veri tipine özgü `Parse()` yöntemi kullanılır.

```
Console.Write("Lütfen yaşınızı giriniz: ");
string girdi = Console.ReadLine();
// string'i int'e dönüştürme (Parse yöntemi)
int yas = int.Parse(girdi);
Console.WriteLine($"Gelecek yıl {yas + 1} yaşında olacaksınız.");
```

§5.4.3 Veri Okuma ve Tip Dönüşümünde Profesyonel Yaklaşım

Eğer kullanıcı sayı yerine harf girerse `int.Parse()`, programı çöktecektir, ancak `TryParse` güvenli bir çıkış sunar.

```
Console.Write("Bir sayı girin: ");
if (int.TryParse(Console.ReadLine(), out int sonuc)) {
    Console.WriteLine($"Girdiğiniz sayının karesi: {sonuc * sonuc}");
} else {
    Console.WriteLine("Hatalı giriş! Lütfen sadece sayı giriniz.");
}
```

§5.4.4 Klavyeden Basılan Tuşu Okuma

Örnek olarak klavyeden girilen karakteri ekranda göstermeden (yıldız basarak) veri okumak isterseniz `ReadKey()` kullanmalısınız.

```
Console.Write("Şifre: ");
ConsoleKeyInfo tus = Console.ReadKey(true); // true argümanı karakteri ekrana yazma demektir.
char karakter = tus.KeyChar; //Klavyeden basılan tuşu öğrendik.
```

§5.4.5 Konsoldan Veri Okurken Neye Dikkat Etmeli?

- ◇ Kullanıcı hiçbir şey yazmadan Enter tuşuna basarsa `Console.ReadLine()` ne döndürür? `null` mu yoksa boş bir metin ("") mı? Bu durum `int.Parse()` yöntemini nasıl etkiler?
- ◇ C#10+ ile gelen boş olabilir referans tip (**nullable reference types**) tanımıyla bu yöntem artık `string?` yani boş olabilir metin olarak işaretlenmiştir. İleride anlatılacaktır.

5.5 Konsola Yazılan Değerleri Biçimlendirme

§5.5.1 Sayı Biçimlendirme

C#6+ sonrasında programlarında ham veriyi olduğu gibi yazdırmak yerine, son kullanıcının anlayabileceği bir forma sokmak (para birimi, yüzde veya belirli bir hassasiyet derecesi gibi) hayati önem taşır. Bu işlem için en modern yöntem metin interpolasyonu (\$) içinde kullanılan **biçim belirleyicilerdir** (**format specifier**).

Tam sayılarda genellikle başına sıfır ekleme veya para birimi gibi gösterimler tercih edilir;

1. **D[n]**: Belirtilen genişliğe kadar başına 0 ekler. `int i=42;` için `{i:D5}` konsola `00042` yazar.
2. **C[n]**: Yerel para birimi sembolünü ekler. `int i=42;` için `{i:C2}` konsola `$42,00` yazar.
3. **R**: C#7.0+ ile birlikte sayının metne dönüştürülüp tekrar sayıya çevrildiğinde değer kaybı yaşanmamasını sağlayan "tam hassasiyet" formatıdır.

```
double a = 0.1;
double b = 0.2;
double toplam = a + b;
Console.WriteLine($"Varsayılan: {toplam}"); // Çıktı: 0,3
Console.WriteLine($"Round-trip: {toplam:R}"); // Çıktı: 0.300000000000000004
```

4. **X**: Sayıyı onaltılık tabana çevirir. `int i=254;` için `{i:X}` konsola `FE` yazar.

Kayan noktalı sayılarda en büyük ihtiyaç, virgülden sonraki basamak sayısını kontrol etmektir. Bunu üç şekilde yapabiliriz;

1. **F[n]**: Sabit basamak sayısı. `float f=12.765F;` için `{f:F2}` konsola `12.77` yazar.
2. **N[n]**: Binlik ayraçlı gösterim. `float f=12.765F;` için `{f:N1}` konsola `12.8` yazar.
3. **P[n]**: Yüzde gösterimi. Sayıyı 100 ile çarpar ve % ekler. `float f=0.25F;` için `{f:P0}` konsola `%25` yazar.
4. **0.##**: Özel Biçimlendirme. Standart formatlar yetersiz kalırsa, kendi şablonunuzu oluşturabilirsiniz.

Özel biçimlendirmede;

- ◊ **0** (Sıfır): Basamakta rakam yoksa mutlaka 0 koyar. `float f=12.7F;` için `{f:0.00}` konsola `12.70` yazar.
- ◊ **#** (Kare): 0 basamakta rakam varsa koyar, yoksa boş bırakır. `float f=12.7;` için `{f:0.##}` konsola `12.7` yazar.

```
double maas = 25500.758;
int kod = 123;
Console.WriteLine($"Maaş Bilgisi (Para Birimi): {maas:C2}"); //$25,500.76
Console.WriteLine($"Maaş Bilgisi (Sade): {maas:F2}"); //25500.76
Console.WriteLine($"Ürün Kodu (5 Haneli): {kod:D5}"); //00123
float f = 12.7654765476547F;
// String Interpolation kullanarak iki hane (precision) sınırlaması:
// 'F2' -> Fixed-point format, virgülden sonra 2 hane.
Console.WriteLine($"Sayının iki haneli hali: {f:F2}"); //12.77
// Alternatif olarak '#' kullanımı (varsa gösterir, yoksa boş bırakır):
Console.WriteLine($"Alternatif biçimlendirme: {f:0.##}"); //12.77
```

Bahsedilen biçimlendirmeler veri tiplerinin `ToString()` yönteminde de kullanılır;

```
float oran = 0.752f;
int sayi = 255;
string sonuc1 = oran.ToString("P1"); // %75,2 (Yüzde formatı)
string sonuc2 = sayi.ToString("D5"); // 00255 (5 haneye tamamla)
string sonuc3 = sayi.ToString("X"); // FF (Onaltılık taban)
```

Metin **enterpolasyonu olmadan** verileri belirli bir sütun genişliğine yayabilir ve sağa/sola yaslayabilirsiniz. Bu, özellikle konsolda tablo oluştururken hayat kurtarır.

```
// Pozitif sayı (10): 10 karakter genişliğinde SAĞA yasla
// Negatif sayı (-10): 10 karakter genişliğinde SOLA yasla
Console.WriteLine("{0,-15} | {1,10}", "Ürün Adı", "Fiyat");
Console.WriteLine("{0,-15} | {1,10:C2}", "Elma", 15.50);
Console.WriteLine("{0,-15} | {1,10:C2}", "Bilgisayar", 25000.00);
/*Program Çıktısı:
Ürün Adı          |      Fiyat
Elma              |      $15.50
Bilgisayar        | $25,000.00
*/
```

Ancak burada izleyen argüman için **yanlış indis numarası verme riski vardır!** Metin enterpolasyonu her za-

man daha iyiye, neden hala `string.Format()` veya indisli yapıyı bilmeliyiz? Bazı durumlarda biçimlendirme taslağı (**template**) dışarıdaki bir dosyadan veya veritabanından gelebilir. Örneğin: "Merhaba 0, bugün hava 1." gibi bir taslağınız varsa, çalışma anında (**run-time**) bu boşlukları doldurmak için `string.Format()` kullanmak zorundasınızdır. Çünkü enterpolasyon derleme anında (**compile-time**) çözülür.

Para birimi biçimlendiricisi olarak `C[n]` **biçim belirleyicisi** (**format specifier**) kullanılır. Bir sayıyı para birimi miktarını temsil eden bir metne (**string**) dönüştürür;

```
using System.Globalization;
double para = 2510.752525;
//Sistem tarafından ön tanımlı olarak bozukluk iki hane yazılır.
//Programın çalıştığı bilgisayar türkçe ayarlarına sahip:
Console.WriteLine(string.Format("{0:c}", para)); // $2.510,75
Console.WriteLine($"{para:C}"); // $2.510,75
//Bozukluk hane sayısı aşağıdaki şekild belirtilir:
Console.WriteLine(string.Format("{0:c1}", para)); // $2.510,8
Console.WriteLine(string.Format("{0:c2}", para)); // $2.510,75
Console.WriteLine(string.Format("{0:c3}", para)); // $2.510,753
Console.WriteLine(string.Format("{0:c4}", para)); // $2.510,7525
Console.WriteLine();

NumberFormatInfo nfi = new CultureInfo("en-US", false).NumberFormat;
nfi.CurrencySymbol = "€";
nfi.NumberGroupSeparator = ",";
nfi.NumberDecimalSeparator = ".";
nfi.CurrencyPositivePattern = 0;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //€2,510.75
nfi.CurrencyPositivePattern = 1;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //2,510.75€
nfi.CurrencyPositivePattern = 2;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //€ 2,510.75
nfi.CurrencyPositivePattern = 3;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //2,510.75 €
```

Ayrıca bölgesel ayarları `NumberFormatInfo` kullanılarak hem tam sayı, hem de gerçek sayıları özel ayrıçlarla biçimlendirmek için kullanılabilir.

```
using System.Globalization;
double number = 1234567.89;
var invarianResult = string.Format(CultureInfo.InvariantCulture, "{0:#,###,##}", number);
Console.WriteLine(invarianResult); // 1,234,568
var customProvider = new NumberFormatInfo {
    NumberDecimalSeparator = "*",
    NumberGroupSeparator = "?",
};
var customResult = string.Format(customProvider, "{0:#,###.##}", number);
Console.WriteLine(customResult); // 1?234?567*89
```

§5.5.2 Zaman Biçimlendirme

Para birimi biçimlendiricisi olarak `d`, `M`, `y`, `H`, `m` ve `s` **biçim belirleyicileri** (**format specifier**) kullanılır. Aşağıda hem enterpolasyonlu, hem de enterpolasyon olmadan kullanım örneği verilmiştir;

```
using System.Globalization;
//Yıl-Ay-Gün-Saat-Dakika-Saniye Şeklinde Zamanı oluşturuyoruz.
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);

Console.WriteLine(String.Format("{0:dd}", date)); //Yalnızca Günü Yazar:01
//Bunun Kısa Hali:
Console.WriteLine($"{date:dd}");
Console.WriteLine("{0:dd}", date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy}", date)); //Günü Ayı ve Yılı Yazar: 01.09.1971
//Bunun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy}");
Console.WriteLine("{0:dd/MM/yyyy}", date);
```



```

Console.WriteLine(String.Format("{0:dd/MM/yyyy HH:mm:ss}", date)); //Günü Ayı Yılı Saat Dakika ve
→ Saniyeyi Yazar: 01.09.1971 11:53:24
//Bunun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy HH:mm:ss}");
Console.WriteLine("{0:dd/MM/yyyy HH:mm:ss}",date);

//Günü İsim Olarak Almanca Yazmak için:
//Gün İsmi Almanca Yazar: Mittwoch
Console.WriteLine(String.Format(new System.Globalization.CultureInfo("de-DE"), "{0:dddd}",
→ date));

```

Aşağıda zaman biçimlendirmek için enterpolasyon işlecine bir örnek verilmiştir;

```

DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
Console.WriteLine($"{date:d}"); //Günü Yazar: 1.09.1971
Console.WriteLine($"{date:dd}"); //Günü Yazar: 01
Console.WriteLine($"{date:ddd}"); //Kısa Günü Yazar: Çar
Console.WriteLine($"{date:dddd}"); //Uzun Günü Yazar: Çarşamba
Console.WriteLine($"{date:D}"); //Uzun Tarih Yazar: 1 Eylül 1971 Çarşamba
Console.WriteLine($"{date:f}"); //Uzun Tarih ve Kısa Saati Yazar: 1 Eylül 1971 Çarşamba 11:53
Console.WriteLine($"{date:F}"); /* Uzun Tarih ve Uzun Saati Yazar:
1 Eylül 1971 Çarşamba 11:53:24 */
Console.WriteLine($"{date:g}"); //Kısa Tarih ve Kısa Saati Yazar: 1.09.1971 11:53
Console.WriteLine($"{date:hh}"); //Saatin 12 Saatlik Formatı: 11
Console.WriteLine($"{date:mm}"); //Dakika Yazar: 53
Console.WriteLine($"{date:MM}"); //Ayı Sayısal Olarak Yazar: 09
Console.WriteLine($"{date:MMM}"); //Ayı Kısa Olarak Yazar: Eyl
Console.WriteLine($"{date:MMMM}"); //Ayı Uzun Olarak Yazar: Eylül
Console.WriteLine($"{date:ss}"); //Saniye Yazar: 24
Console.WriteLine($"{date:r}"); //RFC1123 Formatında Yazar: Wed, 01 Sep 1971 11:53:24 GMT
Console.WriteLine($"{date:t}"); //Kısa Saati Yazar: 11:53
Console.WriteLine($"{date:T}"); //Uzun Saati Yazar: 11:53:24
Console.WriteLine($"{date:tt}"); //AM/PM Formatında Yazar: ÖÖ
Console.WriteLine($"{date:y}"); //Ay ve Yılı Yazar: Eylül 1971
Console.WriteLine($"{date:yy}"); //Yılı Kısa Olarak Yazar: 71
Console.WriteLine($"{date:yyyy}"); //Yılı Uzun Olarak Yazar: 1971

```

5.6 Düşün ve Çöz

- ♦ **Düşün:** `Console.Write()` ile `Console.WriteLine()` arasındaki farkı yalnızca teknik açıdan değil, kullanıcının deneyimi açısından da değerlendiriniz. Bir CLI aracı tasarlarlarken çıktı biçimlendirmesinin önemi nedir?
- ♦ **Düşün:** C#’ta metin enterpolasyonu (`$"degisken"`), `string.Format()` ve `+` ile birleştirmenin performans farkları nelerdir? Büyük döngülerde hangisi tercih edilmeli ve neden?
- ♦ **Düşün:** Kullanıcıdan konsol üzerinden sayısal veri okurken `int.Parse()`, `int.TryParse()` ve `Convert.ToInt32()` arasındaki fark nedir? Hangisi daha ‘profesyonel’ bir yaklaşımdır ve neden?
- ♦ **Düşün:** Kaçış dizileri (`escape sequences`) neden gereklidir? Dosya yollarını `string` olarak yazarken `'\'` yerine `'@'` (`verbatim string`) kullanmanın avantajı nedir?
- ♦ **Düşün:** Para birimi, tarih ve ondalık sayıların biçimlendirilmesinde kültür (`CultureInfo`) ayarlarının önemi nedir? Türkiye’de çalışan bir uygulamanın para gösterimi Almanya’ya taşındığında ne gibi sorunlar çıkabilir?
- ♦ **Düşün:** `Console.ReadLine()` her zaman bir `string` döner. Kullanıcının yanlış tip girmesi durumunu ele almak için hangi stratejiyi benimsersiniz? Sonsuz bir döngüde doğru giriş beklemek ne zaman mantıklıdır?
- ♦ **Düşün:** Metin enterpolasyonunda format belirleyicileri (örneğin `deger:C2` veya `deger:0.00`) kullanmak, `string.Format()` kullanımına kıyasla okunabilirliği nasıl etkiler?
- ♦ **Çöz:** Kullanıcıdan ad, soyad, doğum yılı ve aylık maaşını alan; yaşını hesaplayan ve maaşı hem TL hem de USD (sabit kur kullanarak) olarak gösteren bir program yazınız. Çıktıyı düzgün biçimlendiriniz.
- ♦ **Çöz:** Kullanıcıdan ondalıklı bir sayı girişi isteyen ve hatalı (sayısal olmayan) girişlerde hata mesajı gösterip tekrar soran bir program yazınız. `double.TryParse()` kullanınız.
- ♦ **Çöz:** Tarih ve saat bilgisini `DD/MM/YYYY HH:mm:ss` biçiminde konsola yazan, ardından bu tarihin haftanın hangi gününe denk geldiğini Türkçe olarak gösteren bir program yazınız.

- ◇ **Çöz:** Bir not ortalaması hesap makinesi yazınız: kullanıcı kaç ders girileceğini belirlesin, her dersin adını ve notunu sırasıyla yazsın, program en yüksek, en düşük notu ve ortalamayı düzgün biçimlendirilmiş tablo olarak gösterebilir.
- ◇ **Çöz:** C#'ta `string` sınıfının `Length`, `ToUpper()`, `ToLower()`, `Contains()`, `Replace()` ve `Substring()` yöntemlerini kullanan; kullanıcının girdiği cümleyi çeşitli biçimlerde işleyen bir program yazınız. Soru: "Modüler programlamada 'çağırma yükü' (**calling overhead**) nedir? Modern derleyiciler bu yükü nasıl optimize eder?"
- ◇ **Düşün :** "Bir tarih verisini `Console.WriteLine($"tarih:MMMM")` şeklinde yazdığınızda, ay isminin Türkçe mi yoksa İngilizce mi çıkacağını belirleyen faktör nedir?"